

Clear Description of Procedures/Processes

Staff members + system

- The staff can add an order in the system (with customer details), the system must record which staff member created the order and the timestamp.
- staff member can update the order status to pending (this is the default order status), ready or collect
- staff member can view the average rating their orders have achieved

Customers + system

- Customers are able to rate the service received on each order
- Customers are able to see the average rating given to each restaurant
- Login or view their order status and restaurant information.

Procedures

Procedure 1: staff member adding an order in the system

Actor: system + staff member

- Staff member login into system using valid login credentials
- They will be redirected to the staff dashboard
- Staff add customer details and their staff details (Select Customer Name, Restaurant Name, Staff Name, Time)
- Order status is set to pending by default
- Staff clicks on “add order” button
- The order is added to the database in the orders table

Procedure 2: Customer viewing their order in the system

Actor: system + customer

- Login into system using valid credentials
- Redirected to the customers dashboard
- View their order details, such as order status (pending, ready, collected), order ID, time, staff member that ordered for the customer and restaurant name and restaurant rating

Procedure 3: Staff member updating order in the system

Actor: system + staff member

- Login into system using valid credentials
- Redirected to the staff dashboard
- Click on the order they want to update and change the status to “ready” or “collected”
- Click on the “update order status” button
- The status will be updated in the database
- Change reflects to customer

Procedure 4: customer rating the staff member and restaurant in the system

Actor: system + customer

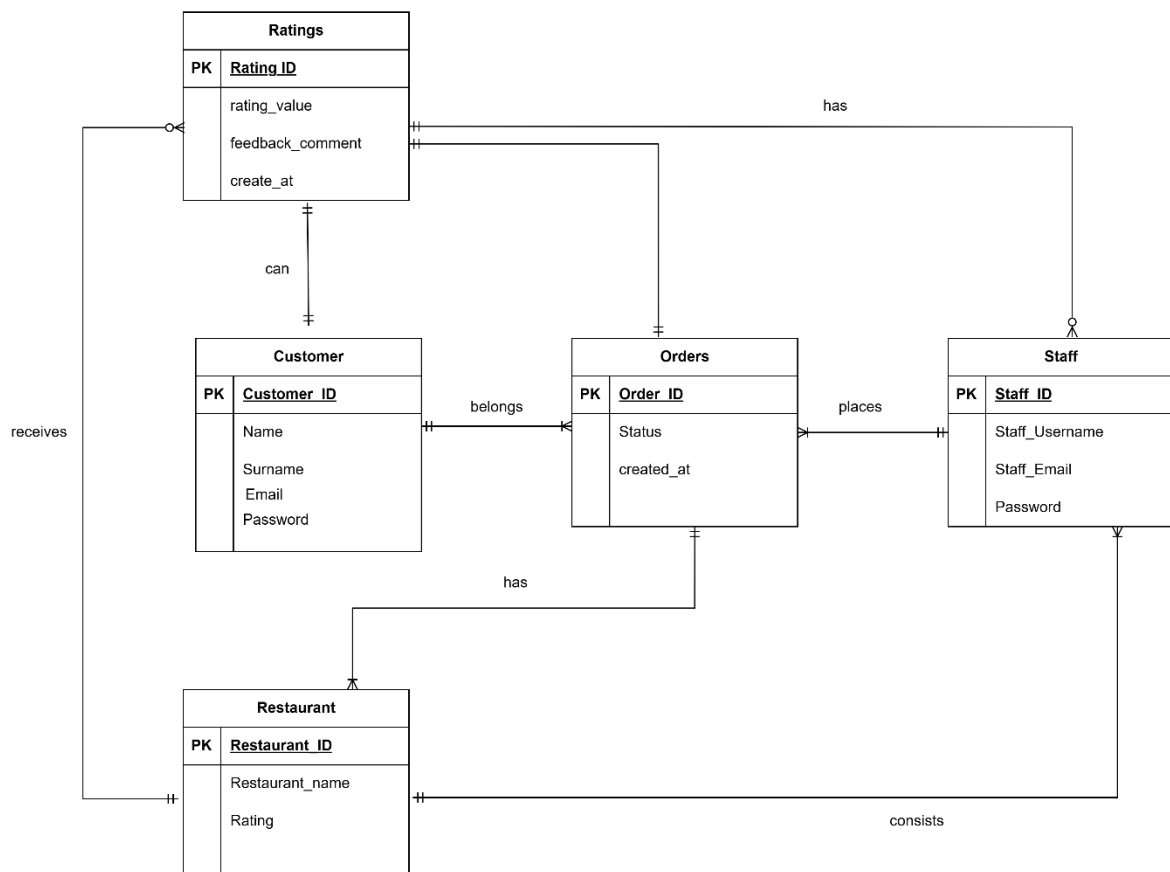
- Login into system using valid credentials
- Redirected into customer dashboard
- Select the order they want to rate
- Click thumb up or down (based on experience)
- Type in the feedback(optional) and submit
- System updates the staff member and restaurant rating

Formulation of Business Rules (Bidirectional)

- Only a staff member can create an order
- A staff member must be registered by an admin before they can log in to the system, and only staff with valid credentials are allowed to access their accounts.
- A customer can rate each order once with either thumbs up or thumbs down.
- A Customer must register first before they can login
- Each order has a status which can be "Pending", "Ready", or "Collected".
- A restaurant can have many orders, and each order is linked to one customer
- A customer can rate many restaurants, and many ratings can belong to a customer
- A Customer can place many orders, and many orders can belong to one customer (app for multiple takeaways not one)
- A staff member can create many orders, and each order is created by one staff member.

- A staff member can receive a rating (via orders), and each rating belong to one staff member
- The average rating Restaurant and Staff is calculated from all ratings of its orders.

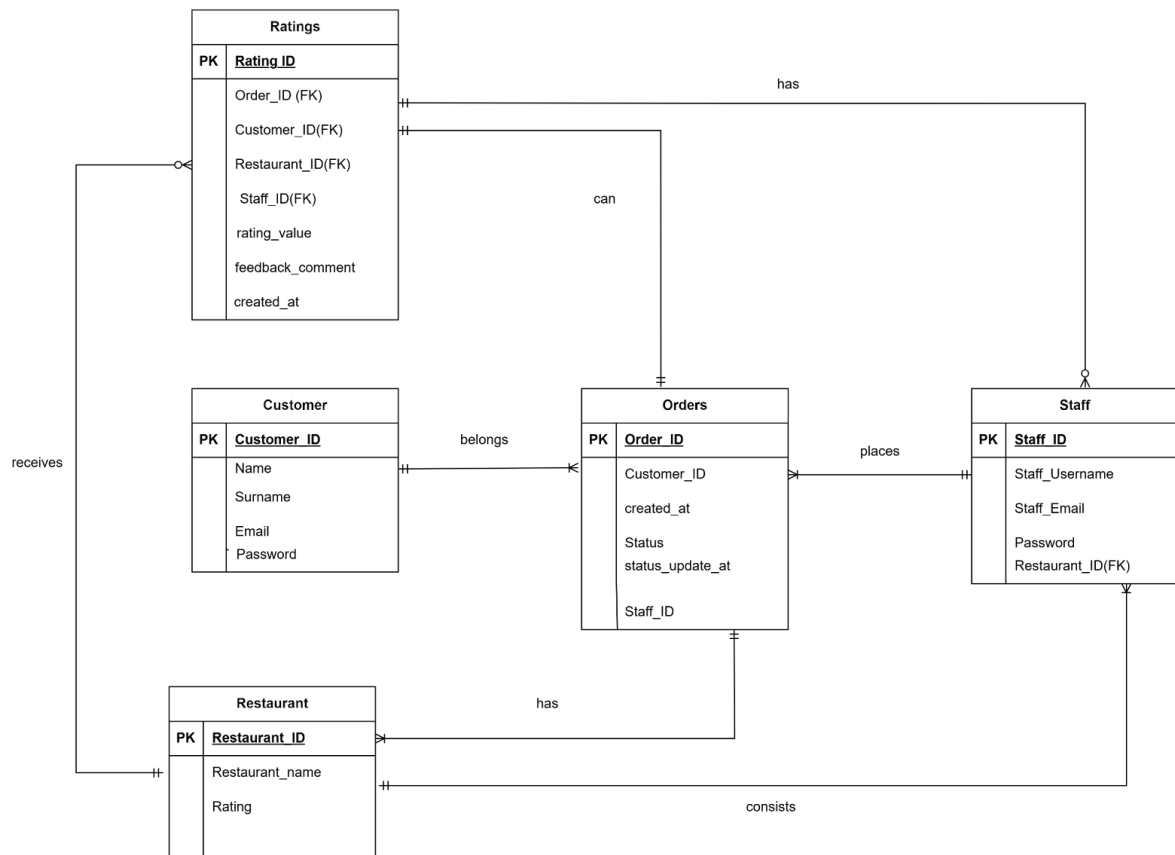
Development of Initial ERD (Entity-Relationship Diagram)



Recognition of Key Issues

- There are no M:N relationships
- For rating, when an order is created the customer order rating won't be available until the order is collected and rated hence, we will have null values
- There are no Multivalued Attribute
- Derived Attributes are calculated from other data in the database. The average rating for restaurant and staff will be calculated from ratings given to the orders.

Solutions Provided & Updated (Final) ERD



Implementation of Tables/Functions/Procedures/Views/Triggers

Restaurant Table

CREATE TABLE Restaurant (Restaurant_ID int NOT NULL AUTO-INCREMENT, Restaurant_name VARCHAR (100) NOT NULL, Rating FLOAT, PRIMARY KEY (Restaurant_ID);

Customer Table

Create TABLE customer2 (Customer_ID VARCHAR (13) NOT NULL, Name VARCHAR (50) DEFAULT NULL, Surname VARCHAR (50) DEFAULT NULL, Email VARCHAR (100) DEFAULT NULL, Password VARCHAR (255) DEFAULT NULL, PRIMARY KEY (Customer_ID), UNIQUE KEY Email (Email));

Staff Table

```
CREATE TABLE Staff (Staff_ID int NOT NULL AUTO-INCREMNT, Staff_Username
VARCHAR (100) NOT NULL, Staff_Email VARCHAR (100) DEFAULT NULL,
Password VARCHAR (255) NOT NULL, Restaurant_ID int DEFAULT NULL,
PRIMARY KEY (Staff_ID) KEY Restaurant_ID (Restaurant_ID), CONSTRAINT
Staff_ibfk_1, FOREIGN KEY (Restaurant_ID) REFERENCES Restaurant
(Restaurant_ID) ON DELETE NULL);
```

Feedback Table

```
CREATE TABLE Feedback (feedbacked int NOT NULL AUTO-INCREMNT,
customerEmail VARCHAR (255) NOT NULL, rating enum ('up', 'down') NOT NULL,
feedbackText text, submittedAt timestamp NULL DEFAULT CURRENT_TIMESTAMP,
PRIMARY KEY (feedbacked);
```

Order Table

```
CREATE TABLE orders (Order_ID int NOT NULL AUTO-INCREMNT, Customer_ID
VARCHAR (13) NOT NULL, Restaurant_ID int NOT NULL, status ENUM
('pending','ready','collected') DEFAULT 'pending', created_at TIMESTAMP NULL
DEFAULT CURRENT_TIMESTAMP, status_updated_at TIMESTAMP NULL
DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, Staff_ID
int NOT NULL, PRIMARY KEY (Order_ID), KEY Customer_ID (Customer_ID), KEY
Restaurant_ID (Restaurant_ID), KEY fk_orders_staff (Staff_ID), CONSTRAINT
fk_orders_staff, FOREIGN KEY (Staff_ID), REFERENCES Staff (Staff_ID),
CONSTRAINT orders_ibfk_1, FOREIGN KEY (Customer_ID) REFERENCES
customer2 (Customer_ID), CONSTRAINT orders_ibfk_2 FOREIGN KEY
(Restaurant_ID) REFERENCES Restaurant (Restaurant_ID));
```

Procedure 1: Staff Adds an Order

Actors: Staff (Frontend) + System (PHP + MySQL)

Frontend Steps:

- Staff logs into the system using valid credentials.
- Redirected to staff dashboard and fill in all required information
- Staff clicks on the “Add Oder: button.

Backend/System Logic:

- The form data is sent to the server as a JSON request via PHP.
- PHP extracts the input data, finds the corresponding Customer_ID, Restaurant_ID Staff_ID and runs an SQL INSERT command.

The status is automatically set to “pending” by default and the timestamp is automatically generated.

Procedure 2: Customer Views Their Order

Actors: Customer + System

Frontend Steps:

- Customer logs in
- Redirected to customer dashboard to view their orders

Backend/System Logic:

- Customer’s ID is set to the server via JSON
- PHP queries the database for all orders where the Customer_ID = ‘X’

The data is returned as JSON and displayed on the customer’s dashboard

Procedure 3: Customer Rates an Order

Actors: Customer + System

Frontend Steps:

- Customer select an order then want to rate
- They choose Thumbs up (1) or Thumbs Down (0)
- Optionally enter feedback text
- Click submit button

Backend/System Logic:

- Customer’s rating and feedback are sent via JSON
- PHP inserts the data into the feedback table and links it to the order
- This feedback is then used to calculate the average rating for: the restaurant and staff member (based on the order they created)

Procedure 4: Staff Updates Order Status

Actors: Staff + System

Frontend Steps:

- Staff logs in
- View the order they created
- Selects an order they want to change to ‘ready’ or ‘collected’
- Clicks on update button

Backend/System Logic:

- The order ID and new status are sent via JSON
- PHP updates the orders table for that order

The updated status reflects in the customer's dashboard